

1

Résumé

Cette étude **améliore l'observabilité d'une application web** en ajoutant une télémétrie frontend à un backend déjà instrumenté avec OpenTelemetry. L'approche retenue permet de corrélér les actions utilisateur et les traitements backend dans .NET Aspire, sans données sensibles ni impact perceptible sur les performances.

Abstract

This study **improves the observability of a web application** by adding frontend telemetry to a backend already instrumented with OpenTelemetry. The chosen approach enables correlation between user actions and backend processing in .NET Aspire, without sensitive data exposure or noticeable performance impact.

4

Problématique & Objectifs

Comment améliorer l'observabilité globale d'une application web en intégrant une télémétrie frontend, afin de permettre une corrélation efficace entre les interactions utilisateur et les traitements backend ?

- Objectifs :
- Instrumenter un **parcours utilisateur** représentatif
 - Identifier une solution compatible avec l'existant OpenTelemetry
 - Vérifier la **corrélation** frontend ↔ backend (même traceId)
 - Valider l'absence de dégradation des performances et de données sensibles

6

Etat de l'art & Solution retenue

L'observabilité repose sur **trois piliers** : logs, métriques et traces distribuées [7].
OpenTelemetry est le standard open source de collecte de télémétrie [1][4], déjà utilisé côté backend dans le projet [4].
Trois approches OpenTelemetry comparées [4][5]:

Critères	Auto	Manuelle	Hybride
Compatibilité backend	✓✓	✓✓	✓✓
Facilité d'intégration	✓✓	-	✓
Lisibilité fonctionnelle	-	✓✓	✓✓
Corrélation front/back	✓	✓✓	✓✓
Maîtrise du périmètre	-	✓✓	✓✓

→ Solution retenue : approche hybride OpenTelemetry

8

Validation & Résultats

- 4 scénarios de test exécutés dans Aspire, un par objectif :
- Collecte** — Spans manuels et automatiques visibles, service fia-production-xxxx identifié dans Aspire
 - Corrélation** — Même traceId partagé entre spans frontend et backend → vue distribuée complète front → BS → CS
 - Confidentialité** — Aucune donnée personnelle dans les attributs des spans
 - Performances** — Aucune dégradation perceptible, instrumentation asynchrone et non bloquante
- Limites** : un seul parcours pilote instrumenté, proxy Vite non transposable en production
- Perspectives** : extension à d'autres parcours, intégration des Web Vitals, collecteur OTel dédié pour la production

9

Conclusion & Perspectives

Cette étude démontre qu'il est possible, **sans modifier l'architecture backend**, d'étendre la chaîne d'observabilité jusqu'au navigateur grâce à une approche hybride OpenTelemetry.

Les 4 objectifs sont atteints : collecte de spans frontend, corrélation de bout en bout via un traceId partagé, respect de la confidentialité et absence de dégradation des performances.

2

Mots clés

- Observabilité
- OpenTelemetry
- Télémétrie Frontend
- Corrélation frontend/backend
- Traces distribuées

3

Introduction & Contexte

Les applications web modernes reposent sur des **architectures distribuées** combinant frontend, API et microservices backend. Dans le projet étudié, l'observabilité est déjà assurée côté serveur grâce à **OpenTelemetry** et **.NET Aspire**, mais aucune télémétrie n'est disponible dans le navigateur. Cette étude vise à étendre cette observabilité jusqu'au **frontend** afin d'obtenir une vision de bout en bout du **comportement de l'application**.

5

Analyse de l'existant

- L'observabilité backend est déjà en place et repose sur **OpenTelemetry [1]** avec une configuration homogène sur les microservices .NET. Ce qui est collecté :
- Traces distribuées** : requêtes HTTP, appels interservices, accès BDD
 - Métriques** : temps de réponse, activité HTTP, runtime .NET, processus
 - Export OTLP [6]** vers le dashboard **.NET Aspire [3]** pour la visualisation
 - Propagation du contexte** de trace (traceparent) entre microservices

- Apports de l'existant :
- Diagnostic technique côté serveur opérationnel
 - Corrélation backend ↔ backend fonctionnelle
 - Socle OTLP déjà exploitable pour l'extension frontend

- Limites identifiées :
- Aucune visibilité côté navigateur
 - Pas de corrélation frontend ↔ backend
 - Données exclusivement techniques :

→ L'observabilité actuelle est utile mais partielle : elle commence trop tard par rapport au parcours réel de l'utilisateur.

7

Mise en oeuvre / Architecture



- Contraintes techniques résolues :
- gRPC non supporté** dans le navigateur → second endpoint OTLP en HTTP/1.1 (port 21012)
 - CORS** → proxy Vite relayant les traces sur la même origine
 - Format** → protobuf imposé (Aspire refuse le JSON)
 - Propagation sélective** → traceparent restreint aux requêtes localhost uniquement

Parcours pilote instrumenté : recherche de lieu de garage par code postal (écran /devis)

Instrumentation automatique : appels fetch GET /api/v1/communes tracés [5] sans code manuel + chargement du document.

4 métriques frontend : fia.page_views, fia.user_actions, fia.errors, fia.api_call_duration

Span	Action utilisateur
garage.popup.open	Ouverture de la popup
garage.search.triggered	Clic sur « Rechercher »
garage.search.results	Réception des résultats
garage.location.confirmed	Validation du choix

- Perspectives :
- Extension de l'instrumentation à d'autres parcours
 - Intégration des Core Web Vitals (LCP, INP, CLS)
 - Collecte des erreurs JavaScript
 - Passage en production avec un collecteur OpenTelemetry dédié

10

Bibliographie

- [1] [CNCF](#). OpenTelemetry
- [2] [Datadog](#). OpenTelemetry Integration.
- [3] [Microsoft](#). .NET Aspire dashboard overview.
- [4] [OpenTelemetry](#). Documentation officielle.
- [5] [OpenTelemetry](#). Getting Started — Browser.
- [6] [OpenTelemetry](#). Schema Specification.
- [7] [Red Hat](#). What is Observability?.